0994. Rotting Oranges

: Medium | Category: Graphs | LeetCode: https://leetcode.com/problems/rotting-

Problem

- 0 empty, 1 fresh, 2 rotten. Each minute, rotten oranges rot their 4-neighbours. Return the minutes until no fresh orange remains, or -1 if impossible.

Intuition

- All rotten oranges spread at the same time: multi-source BFS, where each BFS layer is one minute.

Brute Force

- Simulate minute by minute scanning the whole grid: O((mn)^2).

Optimal Approach

- Enqueue all rotten cells; count fresh.
- BFS level by level; rot fresh neighbours, decrement fresh, enqueue them.
- Minutes = number of levels that rotted something. Return fresh == 0 ? minutes : -1.

Worked Example

```
[[2,1,1],[1,1,0],[0,1,1]].

min 1: (0,1),(1,0)
min 2: (0,2),(1,1)
min 3: (2,1)
min 4: (2,2)          fresh = 0 -> 4
```

Java Solution

```java
public class RottingOranges {
    public int orangesRotting(int[][] g) {
        int m = g.length, n = g[0].length, fresh = 0, minutes = 0;
        Deque<int[]> q = new ArrayDeque<>();
        for (int r = 0; r < m; r++)
            for (int c = 0; c < n; c++) {
                if (g[r][c] == 2) q.add(new int[]{r, c});
                else if (g[r][c] == 1) fresh++;
            }
        int[][] dirs = {{1, 0}, {-1, 0}, {0, 1}, {0, -1}};
        while (!q.isEmpty() && fresh > 0) {
            for (int i = q.size(); i > 0; i--) {
                int[] cur = q.poll();
                for (int[] d : dirs) {
                    int r = cur[0] + d[0], c = cur[1] + d[1];
                    if (r < 0 || c < 0 || r >= m || c >= n || g[r][c] != 1) continue;
                    g[r][c] = 2;
                    fresh--;
                    q.add(new int[]{r, c});
                }
            }
            minutes++;
        }
        return fresh == 0 ? minutes : -1;
    }
}
```

Complexity

- Time: O(m n). Space: O(m n).

Pitfalls

- No fresh oranges at the start → 0, even if there are no rotten ones.